

Markov normal algorithms

Exercise sheet 4

Exercise 4.1.¹ Given a word $w \in \{a, b, c\}^*$, rearrange its letters into nondecreasing order. For example, $cbacba \mapsto aabbcc$.

Solution.

$$ba \rightarrow ab$$

$$ca \rightarrow ac$$

$$cb \rightarrow bc$$

□

Exercise 4.2. The input is o^n . Output the word **odd** when n is odd and **even** when n is even.

Solution.

$$oo \rightarrow$$

$$o \rightarrow. \text{odd}$$

$$\rightarrow. \text{even}$$

□

Exercise 4.3. The input is the standard binary representation of an integer n . Output the binary representation of $n + 1$.

Solution.

$$\#0 \rightarrow 0\#$$

$$\#1 \rightarrow 1\#$$

$$\# \rightarrow *$$

$$0* \rightarrow. 1$$

$$1* \rightarrow *0$$

$$* \rightarrow. 1$$

$$\rightarrow \#$$

□

¹Exercises 1-10 are adapted from <https://mao.snuke.org/tasks/>.

Exercise 4.4. The input is o^n . Output the usual ~~decimal~~ binary representation of n .

Solution. Note that the empty word (zero symbols o) needs to be converted to 0. If we restrict $n > 0$ in the input, the following elegant algorithm solves the problem:

$$\begin{aligned} 0o &\rightarrow 1 \\ 1o &\rightarrow 2 \\ o &\rightarrow 1 \end{aligned}$$

In other words, the first o from the left becomes a 1 (the first two rules can't be applied on the first iteration). This 1 is our counter—now every remaining o will try to add 1 to that counter. Cf. Exercise 4.3: each o behaves like the $*$.

To deal with the case $n = 0$, we add two more rules in the end:

$$\begin{aligned} 0o &\rightarrow 1 \\ 1o &\rightarrow 2 \\ o &\rightarrow 1 \\ 1 &\rightarrow 1 \\ &\rightarrow 0 \end{aligned}$$

That is, after all o s are gone, $1 \rightarrow 1$ fires if the counter is positive, and the execution stops. If there was no o , and thus no 1 appeared, the last rule fires. $\square$

Exercise 4.5. Duplicate a binary word w : $w \mapsto ww$.

Solution. We use $\#$ to scan the word, duplicating every bit we meet (with a hat). Then we send bits with hats to the end — note that the *relative order* between those with the hats is maintained. Then we scan the tail to remove the hats.

$$\begin{aligned} \#0 &\rightarrow 0\hat{0}\# \\ \#1 &\rightarrow 1\hat{1}\# \\ \hat{0}0 &\rightarrow 0\hat{0} \\ \hat{0}1 &\rightarrow 1\hat{0} \\ \hat{1}0 &\rightarrow 0\hat{1} \\ \hat{1}1 &\rightarrow 1\hat{1} \\ \hat{0}\# &\rightarrow \#\hat{0} \\ \hat{1}\# &\rightarrow \#\hat{1} \\ \#\hat{0} &\rightarrow 0\# \\ \#\hat{1} &\rightarrow 1\# \\ \# &\rightarrow . \\ &\rightarrow \# \end{aligned}$$

$\square$

Exercise 4.6. Given a binary word w , output **yes** if w is a palindrome, and **no** otherwise.

Solution (sketch). This algorithm has many rules, so we just describe it. First, use the same idea as in Exercise 4.5 to duplicate the word. This time, however, use two symbols, say $\$$ for the first scan. Then use just $\#$ for the second scan: $\$$ remains in-between the two copies. Then, we check the palindrome property using the rules $0\$0 \rightarrow$ and $1\$1 \rightarrow$. If only $\$$ and $\#$ remain after these rules have been exhaustively applied, the word is a palindrome; otherwise, it is not. Note that we need to clean up the remaining pieces before outputting **no**. $\square$

Exercise 4.7. Reverse a binary word w : e.g. $111001 \mapsto 100111$.

Solution. We scan the word from the right, putting a hat on each bit and making the hatted bits flow left. But this time, we make them run before the scanning symbol:

$$\begin{aligned} \#0 &\rightarrow 0\# \\ \#1 &\rightarrow 1\# \\ \# &\rightarrow \$ \\ 0\$ &\rightarrow \hat{0}\$ \\ 1\$ &\rightarrow \hat{1}\$ \\ 0\hat{0} &\rightarrow \hat{0}0 \\ 0\hat{1} &\rightarrow \hat{1}0 \\ 1\hat{0} &\rightarrow \hat{0}1 \\ 1\hat{1} &\rightarrow \hat{1}1 \\ \hat{0}\$ &\rightarrow \$0 \\ \hat{1}\$ &\rightarrow \$1 \\ \$ &\rightarrow . \\ &\rightarrow \# \end{aligned}$$

$\square$

Exercise 4.8. The input has the form $o^n\#o^m$. Output $o^{\gcd(n,m)}$.

Solution (sketch). A possible solution can use Euclid's algorithm, in its difference-based form:

```
def gcd(n, m):
    while n != m:
        if n > m:
            n = n - m
        else:
            m = m - n
    return n
```

We could copy the input with Exercise 4.5, compare n and m , e.g. with the help of the rule $o\#o \rightarrow \#$, like in Exercise 4.6. Depending on the result of the comparison (say, $n < m$), we go to the original input, duplicate o^n with hats—then the rule $\hat{o}\#o \rightarrow \#$ will perform $m = m - n$. And so on.

□

Exercise 4.9. Evaluate a Boolean expression in prefix Polish notation over $\{0, 1, \wedge, \vee\}$, leaving its value, 0 or 1, as the result.

Solution. Polish notation (prefix or postfix) is a nice thing to learn about—do it if you haven't heard of it before. The solution is simple here:

$$\wedge 00 \rightarrow 0$$

$$\wedge 01 \rightarrow 0$$

$$\wedge 10 \rightarrow 0$$

$$\wedge 11 \rightarrow 1$$

$$\vee 00 \rightarrow 0$$

$$\vee 01 \rightarrow 1$$

$$\vee 10 \rightarrow 1$$

$$\vee 11 \rightarrow 1$$

□

Exercise 4.10. * The input is o^{n^2} . Output o^n .

Solution (sketch). We note that $\sum_{k=1}^n (2k - 1) = 1 + 3 + 5 + \cdots + (2n - 1) = n^2$. So, we create three things: a counter (initially ε), a subtrahend (initially o), and a minuend (initially o^{n^2}), all separated by special symbols. While the minuend is not ε , do the following:

1. chop from the minuend as many o as there are in the subtrahend;
2. append the counter with o ;
3. append the subtrahend with oo .

When the minuend becomes ε , the counter is the answer.

□

Exercise 4.11. * Prove that normal algorithms are Turing-complete. That is, every computable function is computed by some normal algorithm.

Proof (sketch). Clearly, a Turing machine can apply the rules of a given normal algorithm in the right order, so normal algorithms are no more powerful than machines. Now, let M have state

set Q , initial state q_0 , halting state q_h , and transition function δ over the binary alphabet $\{0, 1\}$; we construct a normal algorithm with the rules below, where a ranges over $\{0, 1\}$ and q over $Q \setminus \{q_h\}$:

$\$a \rightarrow a\$$	$\$$ flows to the end
$q\$ \rightarrow q0\$$	for $q \in Q$, at the right end, q is looking at an empty cell
$qa \rightarrow bq'$	$\delta(q, a) = (q', b, R), q \neq q_h$
$cqa \rightarrow q'cb$	$\delta(q, a) = (q', b, L), q \neq q_h$
$\#qa \rightarrow \#q'0b$	$\delta(q, a) = (q', b, L), q \neq q_h$
$qa \rightarrow q'b$	$\delta(q, a) = (q', b, S), q \neq q_h$
$q_h a \rightarrow a$	
$\# \rightarrow$	
$\$ \rightarrow.$	
$\rightarrow \#q_0\$$	initial state + end-markers

Intuitively, the word in the workspace represents the tape contents of the machine, each 0 or 1 standing for a cell with that symbol. If such a cell is preceded by a symbol q , it means that the machine is currently in state q and observing that cell. $\square$